# Project Report: Real-Time Material Classification

## 1. Executive Summary

- Goal: Automatically recognize common materials (glass, paper, cardboard, plastic, metal, trash) from images and live camera.
- Approach: Extract image features, train simple ML models (SVM, kNN), and run real-time prediction with a webcam.
- Result: SVM validation accuracy ≈ 78%; kNN ≈ 77%. Low-confidence cases are labeled as “Unknown”.

## 2. Objectives

- Build a clean, balanced dataset of six material classes.
- Turn images into numeric features suitable for machine learning.
- Train and compare two lightweight classifiers (SVM, kNN).
- Provide a simple real-time app that predicts material class from the camera feed.

## 3. Data

- Location: `data/`

    - `raw/`: original images

    - `cleaned/`: validated images (corrupt files removed, consistent format)

    - `augmented/`: balanced dataset with more images per class

    - `features/`: optional HOG-based feature file (`data.npz`)

- Classes: `glass`, `paper`, `cardboard`, `plastic`, `metal`, `trash`.

## 4. Preprocessing and Augmentation

- Cleaning (`src/clean.py`):

    - Skips corrupt files.

    - Converts images to consistent color format.

    - Saves valid images in a standard format.

- Augmentation (`src/augmentation.py`):

    - Adds more images per class to improve balance and generalization.

    - Uses rotations, flips, and color/brightness changes.

    - Resizes images to a small, consistent size for speed.

## 5. Feature Extraction

- Two feature extraction options exist in this project:

    1) Classical HOG features (batch pipeline)

        - Script: `src/extraction.py`

        - Output: `data/features/data.npz` with `X` (features), `y` (labels), `class_map`.

        - Idea: Convert images to grayscale, resize to 64×64, then compute HOG.

**HOG is fast and explainable but limited in representing complex visual patterns.**

2) CNN features from a pretrained MobileNetV2 (used by models and the real-time app)

- Module: `src/features/extract_features.py`

- Steps: Resize to 224×224, convert BGR→RGB, preprocess, and run MobileNetV2 (ImageNet weights, avg pooling).

- Output: A fixed-length deep feature vector per image/frame.

**CNN features capture high-level semantic information and significantly outperform HOG**

## 6. Building the Training Dataset (CNN features)

- Script: `src/models/build_datasets.py`
- Reads images from `data/augmented/` and runs `extract_features()` (MobileNetV2).
- Saves arrays: `src/models/X.npy` (features) and `src/models/y.npy` (labels).
- Labels correspond to class indices in the folder order.

## 7. Models

- kNN (`src/models/train_knn.py`):

  - Pipeline: `StandardScaler` → `KNeighborsClassifier(n_neighbors=3, weights="distance")`.

  - Trains on `X.npy`, `y.npy`; evaluates on a validation split.

  - Saves to `src/models/saved_models/knn_model.joblib`.

- SVM (`src/models/train_svm.py`):

  - Pipeline: `StandardScaler` → `SVC(kernel="linear", probability=True)`.

  - Trains on `X.npy`, `y.npy`; evaluates on a validation split.

  - Saves to `src/models/saved_models/svm_model.joblib`.

- Unknown handling (`src/models/predict_with_unknown.py`):

  - Uses class probabilities. If the highest probability < 0.6, returns index for "Unknown".

## 8. Real-Time Application

- File: `src/app/realtime_classifier.py`
- Loads the trained SVM model and uses the CNN feature extractor.
- Opens the webcam, extracts features from each frame, predicts the class, and overlays the label on the video.
- Press `q` to quit. If the model is not confident, it displays "Unknown".

## 9. Results

- From `Accuracy.txt`:

- SVM Accuracy ≈ 0.78

- kNN Accuracy ≈ 0.77

- Notes:

- Accuracy can vary based on dataset size, balance, and lighting conditions.

- CNN features generally outperform classical HOG here due to richer representations.

## 10. How to Run

### Environment

- See `requirements.txt` and install dependencies (TensorFlow/Keras, OpenCV, scikit-learn, joblib, numpy, etc.).

### Typical Steps

1) Clean and augment data (optional if already prepared):

```
python src/clean.py

python src/augmentation.py
```

2) Build CNN feature dataset and save `X.npy`, `y.npy`:

```
python src/models/build_datasets.py
```

3) Train models and check accuracy:

```
python src/models/train_svm.py

python src/models/train_knn.py
```

4) Run the real-time classifier:

```
python src/app/realtime_classifier.py
```

### Optional: HOG Feature Pipeline

- If you prefer classical features:

```
python src/extraction.py
```

- This creates `data/features/data.npz`, which you could adapt to train models similarly.

## 11. Folder Overview

- `src/app/`: real-time webcam classifier.
- `src/features/`: CNN feature extraction function.
- `src/models/`: dataset builder, training scripts, saved models, and probability-based unknown handling.
- `src/`: cleaning, augmentation, and HOG extraction scripts.
- `data/`: datasets in different processing stages.
- `report/`: this report.

## 13. Conclusion

- The project delivers a practical, lightweight pipeline for material classification using deep features and simple classifiers.
- It includes both batch processing and real-time prediction, with clear steps to reproduce and extend.